Ниже — переписанный документ в корректном Markdown (заголовки, таблицы, блоки кода, чек-лист).

```
# Переход AccountingPeriodService на «контекст на операцию»

Пошаговое руководство: как убрать поле `_context` из сервиса и перейти на
нормальные подключения — одно на единицу работы.
Документ написан как инструкция для самостоятельной работы; код в примерах —
образец, а не готовый патч.

Дата: 23.10.2026.

## Содержание

1. Цель
2. Исходное состояние файла
3. Шаг 0. Фабрика контекста
4. Шаг 1. Пилот: `EnsureDateCanBeModifiedAsync`
5. Шаг 2. Чтения, возвращающие сущности
6. Шаг 3. `EnsureSchemaAsync`
7. Шаг 4. Малые записи
8. Шаг 5. `CloseAsync` и `CloseModuleAsync`
9. Шаг 6. `CollectAsync` — последним
10. Шаг 7. Точки создания
11. Порядок работ и проверки
12. Подводные камни
13. Оценка
14. Чек-лист
15. Связанные документы

---

## 1. Цель

Сейчас сервис держит один `AppDbContext` в поле и делит его со всем
приложением:

```csharp
public class AccountingPeriodService
{
    private readonly AppDbContext _context;
    public AccountingPeriodService(AppDbContext context) => _context =
```
```

```
context;
}
```

Из-за этого:

- два потока не могут одновременно работать с сервисом — Npgsql не допускает параллельные команды на одном соединении (`A command is already in progress`);
- появился костыль `_ensureSchemaGate` (семафор), который «сериализует» вызовы;
- контекст живёт столько же, сколько окно, и накапливает трекинг.

Цель: контекст создаётся в начале публичного метода и уничтожается в конце. Сервис становится без состояния, гонки исчезают по построению, семафор не нужен.

2. Исходное состояние файла

Показатель	Значение	Что означает
Обращений к `_context`	55	объём правки
`SaveChangesAsync`	11	почти все — внутри `CollectAsync`
Транзакций	0	хорошая новость: сворачивать нечего
Публичных методов	9	девять единиц работы
Методов, возвращающих сущности EF	3	`CollectAsync` , `FindAsync` , `GetModuleStatusesAsync`
Вложенных сервисов на том же контексте	5 вызовов	`BalanceService` (2), `OrganizationBalanceService` , `MetadataService` (2)
Точек создания	8	4 окна, 3 теста, `PostingSourceDocumentOpener`

Публичные методы:

```
EnsureSchemaAsync()                          // DDL + сид строк отчётов
CollectAsync(start, end)                     // сбор периода (много записи)
CloseAsync(periodId)                         // закрытие периода
ReopenAsync(periodId)                        // переоткрытие
FindAsync(start, end)                        // чтение периода
GetModuleStatusesAsync(periodId)             // чтение статусов модулей
CloseModuleAsync(periodId, moduleId)         // закрытие модуля
ReopenModuleAsync(periodId, moduleId)        // переоткрытие модуля
EnsureDateCanBeModifiedAsync(date)           // проверка закрытого периода
```

3. Шаг 0. Фабрика контекста

Без неё начинать нельзя: сервису нужно откуда-то брать контекст, иначе он начнёт ходить в `ServiceLocator` напрямую — то же самое, только спрятанное.

Правильный вариант:

```
// Services/IAppDbContextFactory.cs
public interface IAppDbContextFactory
{
    /// <summary>Контекст текущей информационной базы. Вызывающий обязан
освободить.</summary>
    Task<AppDbContext> CreateAsync();
}

// Services/AppDbContextFactory.cs
public sealed class AppDbContextFactory : IAppDbContextFactory
{
    private readonly InfoBaseManager _infoBaseManager;
    public AppDbContextFactory(InfoBaseManager infoBaseManager) =>
_infoBaseManager = infoBaseManager;

    public Task<AppDbContext> CreateAsync() =>
_infoBaseManager.GetCurrentDbContextAsync();
}
```

Промежуточный вариант, если не хочется вводить интерфейс:

```
public AccountingPeriodService(Func<AppDbContext> contextFactory) =>
_contextFactory = contextFactory;
```

Фабрика не кэширует контекст: каждый вызов — новый объект. Npgsql держит пул соединений, поэтому это дешёвая операция, а не новое TCP-подключение.

4. Шаг 1. Пилот: `EnsureDateCanBeModifiedAsync`

Самый безопасный метод — чистое чтение, ничего не пишет.

Было:

```
public async Task EnsureDateCanBeModifiedAsync(DateTime date)
{
    var closed = await _context.AccountingPeriods
        .AsNoTracking()
```

```
        .AnyAsync(period => period.IsLocked && date >= period.StartDate &&
date <= period.EndDate);
    ...
}
```

Стало:

```
public async Task EnsureDateCanBeModifiedAsync(DateTime date)
{
    await using var db = await _factory.CreateAsync();

    var closed = await db.AccountingPeriods
        .AsNoTracking()
        .AnyAsync(period => period.IsLocked && date >= period.StartDate &&
date <= period.EndDate);
    ...
}
```

Проверка: сборка + открытие окна «Закрытие периодов». Если этот метод работает — механика перехода подтверждена.

5. Шаг 2. Чтения, возвращающие сущности

`FindAsync` и `GetModuleStatusesAsync`.

```
public async Task<AccountingPeriod?> FindAsync(DateTime startDate, DateTime
endDate)
{
    await using var db = await _factory.CreateAsync();

    return await db.AccountingPeriods
        .AsNoTracking()
        .FirstOrDefaultAsync(period => period.StartDate == startDate &&
period.EndDate == endDate);
}
```

Важное решение. Сущность уедет в окно отсоединённой:

- читать поля можно;
- навигационные свойства не подгрузятся;
- `SaveChanges` по ней больше не сработает — молча ничего не сохранится.

Если окно только показывает данные — `AsNoTracking()` достаточно. Если правит — возвращайте DTO (`Id` , `StartDate` , `EndDate` , `Status`) и перечитывайте период в новом

контексте при сохранении.

6. Шаг 3. `EnsureSchemaAsync`

DDL из этого метода уже вынесен в шаг миграций `2026.10.19.07` (`SchemaMigrationService.EnsureAccountingPeriodSchemaAsync`). Остаётся:

- убрать из метода прямой DDL;
- выполнить `SeedFinancialReportLinesAsync` на своём контексте;
- удалить `_ensureSchemaGate` — семафор существовал только потому, что все работали через один `AppDbContext` . С отдельными контекстами он не нужен.

```
public async Task EnsureSchemaAsync()
{
    await using var db = await _factory.CreateAsync();

    await SchemaMigrationService.EnsureAccountingPeriodSchemaAsync(db);
    await SeedFinancialReportLinesAsync(db);
}

private async Task SeedFinancialReportLinesAsync(AppDbContext db) { ... }
```

7. Шаг 4. Малые записи

`ReopenAsync` , `ReopenModuleAsync` :

```
public async Task ReopenAsync(Guid periodId)
{
    await using var db = await _factory.CreateAsync();

    var period = await db.AccountingPeriods.FindAsync(periodId)
        ?? throw new InvalidOperationException("Период не найден.");

    period.Status = AccountingPeriodStatus.Open;
    period.ClosedAt = null;
    period.UpdatedAt = DateTime.UtcNow;

    await db.SaveChangesAsync();
}
```

Правила:

- контекст открыт в начале метода, `SaveChanges` — в том же методе;

- помощники получают `db` параметром: `private async Task Helper(AppDbContext db, ...);`
- никаких полей класса для контекста.

8. Шаг 5. `CloseAsync` и `CloseModuleAsync`

То же самое, но внимательнее: после замены `_context` → `db` компилятор покажет все места, где контекст использовался в помощниках. Это и есть чек-лист: каждую ошибку компиляции закрываем передачей `db` параметром.

9. Шаг 6. `CollectAsync` — последним

Самый тяжёлый метод: 11 `SaveChanges`, вызовы `BalanceService`, `OrganizationBalanceService`, `MetadataService`.

Хорошая новость: эти сервисы принимают контекст в конструкторе, поэтому достаточно, чтобы `db` был локальной переменной:

```
public async Task<AccountingPeriod> CollectAsync(DateTime startDate,
DateTime endDate)
{
    await using var db = await _factory.CreateAsync();

    var balances = await new
BalanceService(db).GetTurnoverBalanceAsync(startDate, endDate);
    await new OrganizationBalanceService(db).CalculateAsync(startDate,
endDate);
    var metadataService = new MetadataService(db);

    // ... остальная работа, все SaveChanges — на этом же db
}
```

Переписывать эти три сервиса не нужно: они получат ваш контекст и будут работать в рамках одной единицы работы. В этом и смысл подхода — контекст принадлежит операции, а не классу.

Отдельно про возвращаемое значение. Сейчас метод отдаёт `AccountingPeriod`. После закрытия контекста это отсоединённая сущность. Лучше вернуть DTO:

```
public sealed record AccountingPeriodResult(
    Guid Id, DateTime StartDate, DateTime EndDate, string Status, DateTime?
ClosedAt);
```

Окну она нужна только для отображения; DTO исключает попытку сохранить её «как есть».

Про 11 `SaveChanges`. После перехода стоит свести их к одному-двум осознанным: сейчас при ошибке на середине половина работы уже сохранена, и период остаётся в несогласованном состоянии.

10. Шаг 7. Точки создания

Восемь мест, в каждом меняется одна строка:

Файл	Что сделать
`Views/AccountingSetupView.xaml.cs`	`new AccountingPeriodService(Se`
`Views/AccountingReportsView.xaml.cs`	то же
`Views/PeriodClosingView.xaml.cs`	то же
`Views/PostingSourceDocumentOpener.cs`	то же
`BIS.ERP.TESTS.Core/FixedAssetsSmokeScenario.cs`	фабрика поверх тестового кон
`BIS.ERP.TESTS.Core/FinancePeriodClosingScenario.cs`	то же
`BIS.ERP.TESTS.Core/FinanceReportsScenario.cs`	то же
`BIS.ERP.TESTS.Core/FinanceRegistersScenario.cs`	то же

В тестах проще всего сделать локальную реализацию фабрики, отдающую новый контекст на той же строке подключения.

11. Порядок работ и проверки

Шаг	Метод	Что проверить
1	`EnsureDateCanBeModifiedAsync`	сборка, открытие «Закрытия периодов»
2	`FindAsync`, `GetModuleStatusesAsync`	список периодов, статусы модулей
3	`EnsureSchemaAsync`	запуск без `A command is already in progress`
4	`ReopenAsync`, `ReopenModuleAsync`	переоткрытие периода и модуля
5	`CloseAsync`, `CloseModuleAsync`	закрытие модуля и периода
6	`CollectAsync`	полный сбор периода, проводки, остатки

Финальная проверка — сценарий `FinancePeriodClosingScenario` из тестового проекта: он проходит цикл «собрать → закрыть → переоткрыть».

В логе не должно быть:

- `A second operation was started on this context instance…`
- `A command is already in progress…`
- `ObjectDisposedException`

12. Подводные камни

Ситуация	Как правильно
Метод возвращает `AccountingPeriod`, окно правит и вызывает `SaveChanges`	вернуть DTO или `Id`, перечитать в новом контексте
Помощник обращается к `_context`	передать `db` параметром
Цепочка «загрузил → позже сохранил» в разных методах	свернуть в один метод с одним контекстом
Нужна транзакция	`await using` + `BeginTransaction` / `Commit` в одном методе (здесь транзакций нет)
`_ensureSchemaGate`	удалить на шаге 3 — он больше ничего не защищает
`MetadataService` внутри	на первом этапе работает на вашем `db`, отдельно его не трогаем
11 `SaveChanges` в `CollectAsync`	свести к одному-двум, иначе возможна частично сохранённая работа

13. Оценка

Работа	Время
Фабрика контекста (шаг 0, на весь проект)	1 день
Сам сервис	0,5–1 день
Окна и тесты (8 точек)	0,5 дня
Итого на `AccountingPeriodService`	1–1,5 дня (при готовой фабрике)

Ориентир для остальных: если держаться этой схемы, 44 сервиса проходятся быстрее, чем 4–5 недель из общего плана, — за счёт того, что крупные сервисы (`MetadataService`,

`ReportDataSetService`) можно оставить напоследок и работать по отработанному шаблону.

14. Чек-лист

- [ ] `IAppDbContextFactory` создан и зарегистрирован
- [ ] `AccountingPeriodService` принимает фабрику, поля `_context` нет
- [ ] `EnsureDateCanBeModifiedAsync` переведён
- [ ] `FindAsync`, `GetModuleStatusesAsync` переведены, сущности отсоединены осознанно
- [ ] `EnsureSchemaAsync` без DDL, `_ensureSchemaGate` удалён
- [ ] `ReopenAsync`, `ReopenModuleAsync` переведены
- [ ] `CloseAsync`, `CloseModuleAsync` переведены
- [ ] `CollectAsync` переведён, `SaveChanges` сведены, возвращается DTO
- [ ] 8 точек создания обновлены
- [ ] `FinancePeriodClosingScenario` проходит
- [ ] В логе нет `A command is already in progress` и `ObjectDisposedException`

15. Связанные документы

Документ	Что внутри
`Docs/План_контекст_на_операцию.md`	общий план перехода, волны, риски, состояние на 23.10.2026
`Docs/Что_сделано_2026-10-23.md`	что уже сделано: миграции, кэш схемы, отметки посадок, замеры
`Docs/Задержки_запуска_и_оптимизация.md`	разбор задержек запуска по VPN и замеры

```
Если нужен именно файл (например, `Docs/
Переход_AccountingPeriodService.md`), скажите путь — сохраню.
```